

Kinematics Engine & Vehicle Control Specification

Technical Guide to Stopping Distance Envelopes, Required Deceleration, and Control Arbitration

Metric / Scope	Specification / Target
Subsystem: Kinematics & Vehicle Control	Module Path: core_safar_logic/c_core & core_models
Reaction Time: 0.18s	Emergency Deceleration: 8.5 m/s^2
Steering Authority: 100% Driver Owned	Execution Speed: Sub-millisecond (< 0.1 ms)

1. Subsystem Architecture & Longitudinal Physics

The Kinematics and Control Subsystem computes the deterministic physical boundaries governing vehicle motion. It operates in both high-performance C++20 ('c_core') and Python ('core_models'), calculating stopping envelopes, required deceleration gradients, and control override arbitration.

2. Fundamental Kinematic Equations

- Total Metric Stopping Distance:** $d_{\text{stop}} = v_{\text{ego}} \cdot t_{\text{react}} + \frac{v_{\text{ego}}^2}{2 \cdot a_{\text{decel}}}$
- Required Deceleration:** $a_{\text{req}} = \frac{v_{\text{ego}}^2 - v_{\text{target}}^2}{2 \cdot d_{\text{forward}}}$
- Normalized Brake Authority:** $\text{Brake} = \text{clamp}\left(\frac{a_{\text{req}}}{a_{\text{max}}}, 0.0, 1.0\right)$
- Physical Constants:** $t_{\text{react}} = 0.18 \text{ s}$, $a_{\text{nominal}} = 6.0 \text{ m/s}^2$, $a_{\text{emergency}} = 8.5 \text{ m/s}^2$.

3. Source Code Files Breakdown

File Name	Responsibility & Architectural Work
c_core/include/safar/threat_engine.hpp	Declares native C++ threat calculations. Inlines stopping distance formulas and TTC math for sub-microsecond embedded execution.
c_core/src/threat_engine.cpp	Implements 'SAFARThreatEngine::EvaluateThreat()'. Zero-allocation C++ loop computing distance ratios and required deceleration.
c_core/include/safar/geometry.hpp	Declares vector mathematics, coordinate transformations, and ego corridor intersection geometry.
core_models/models.py	Declares core data structures: 'VehicleState' (velocity, heading, acceleration), 'Obstacle', 'ActionType', 'Decision'.
core_models/vehicle/safety_override.py	Implements pedal override arbitration: clamps throttle to 0.0 during override and blends manual and automated brake inputs.

4. Line-by-Line Code Walkthrough: Native C++ Stopping Math

```
float USAFARThreatEngine::CalculateStoppingDistance(float SpeedMps, float DecelMps2) const
{
    // Reaction distance: distance covered during brake system hydraulic latency (180ms)
    const float ReactionDist = SpeedMps * ReactionTimeSeconds;
    // Braking distance: work-energy theorem d = v^2 / (2 * a)
    const float BrakingDist = (SpeedMps * SpeedMps) / (2.0f * FMath::Max(DecelMps2, 1.0f));
    return ReactionDist + BrakingDist + SafetyMarginMeters;
}
```

Code Explanation: Line 4 models vehicle displacement during sensor-to-actuator latency (180ms). Line 6 calculates kinematic stopping distance under constant deceleration. Line 7 adds a metric safety buffer (3.0m).